

## 第十章 提示学习

经过预训练、指令微调和人类对齐后，我们接下来讨论如何通过提示学习方法来有效地使用大语言模型解决实际任务。目前常用的方法是设计合适的提示（Prompting），通过自然语言接口与大模型进行交互（第 10.1 节）。在现有研究中，任务提示的设计主要依靠人工设计和自动优化两种策略来实现。为了更好地解决未见过的任务，一种典型的提示方法是上下文学习（In-context Learning, ICL），它将任务描述与示例以自然语言文本形式加入到提示中（第 10.2 节）。此外，思维链提示（Chain-of-Thought, CoT）作为一种增强技术，将一系列中间推理步骤加入到提示中，以增强复杂推理任务的解决效果（第 10.3 节）。

### 10.1 基础提示

因为大语言模型的微调代价较高，基于自然语言的提示方法已经成为了使用大语言模型解决下游任务的主要途径。由于提示的质量在很大程度上会影响大语言模型在特定任务中的表现，因此一系列工作深入研究了通过人工设计或自动优化的方法来生成合适的任务提示。本节将对这两种提示方法进行详细的介绍。

#### 10.1.1 人工提示设计

针对特定任务设计合适的任务提示，这一过程被称为“提示工程”（Prompt Engineering）。在本节中，我们将首先介绍构成提示的关键要素，随后介绍人工设计提示的重要基本原则，并提供一些相关的实用建议和案例。读者可以进一步参考相关论文 [263] 和网站<sup>1</sup>，以获得更为全面的提示设计建议。

##### 关键要素

一般而言，针对大语言模型的提示设计需要考虑四个关键要素，即任务描述、输入数据、上下文信息和提示策略。下面将对这四个关键要素进行具体介绍。

- **任务描述**. 任务描述部分展示了大语言模型应当遵循的具体指令。一般来说，用户应该使用清晰的、具体的表达来描述任务目标。进一步，某些特定任务还需

---

<sup>1</sup><https://platform.openai.com/docs/guides/gpt-best-practices>

要对于输入或输出的格式进行更详细的说明，可以使用关键词或者特殊符号来强调特殊设置以指导大语言模型更好地完成任务。

**知识问答的任务描述：**请使用所提供的以三个井号（###）分隔的文章回答问题。如果在文章中找不到答案，请回答“无法找到答案。”

**代码补全的任务描述：**你是一名程序员。给你一个代码片段，你的目标是完成这段代码，确保它能实现描述的功能。

**对话推荐的任务描述：**推荐 10 个符合用户偏好的商品。推荐列表可以包含对话框之前提到的商品。推荐列表的格式为：商品 ID 标题（年份）。请勿在推荐列表中提及商品标题以外的任何内容。

### 例 10.1 提示设计中任务描述的例子

- 输入数据. 通常情况下，用户可以直接使用自然语言描述输入数据的内容。对于特殊形式的输入数据，则需要采用合适的方法使其能够被大语言模型读取与理解。例如，对于结构化数据（如知识图谱、表格等），通常使用线性化方法将其转换为易于处理的文本序列 [264]。此外，由于结构化数据具有较好的组织形式，可以使用编程代码中的数据结构进行存储表示，将结构化数据中的属性（例如图上的节点）表示为数据结构中的变量。基于代码表示的结构化数据可以使用外部工具（如程序执行器）进行准确地读取。

**线性化后的表格数据：**

### 学号 姓名 成绩 # 01278 张三 89.0 # 03813 李四 92.5 # 06714 王五 79.0 ###

**代码格式的图数据：**

```
Graph[name="Knowledge-Graph"]{
  entity_list=["James Cameron", "Ontario", ...],
  triple_list=[("James Cameron" -> "Ontario")[relation="born in"], ...],
}
```

### 例 10.2 提示设计中输入数据的例子

- 上下文信息. 除了任务描述和输入数据外，上下文信息对某些特定任务也非常重要。例如，搜索引擎可以为开放问答任务提供参考文档，可以通过将检索到的参考文档以上下文信息的形式引入提示作为大语言模型的输入。在引入外部信息

时，需要对于这些信息进行合适的格式化，以加强大语言模型对它们的利用。此外，上下文学习（参考第 10.2 节）中的任务示例数据也有助于提升大语言模型处理复杂任务的能力，大模型可以通过这些示例数据学习任务目标、输出格式以及输入和输出之间的映射关系。

**基于任务示例回答问题：**

**示例问题：**罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？

**示例回答：**罗杰一开始有 5 个球，2 罐每罐 3 个网球就是 6 个网球。 $5 + 6 = 11$ 。答案是 11。

**用户输入：**袋子里有 16 个球，其中一半是高尔夫球，并且所有高尔夫球中有一半是蓝色的球。请问有多少个蓝色高尔夫球？

**模型输出：**袋子里有 16 个球，其中一半是高尔夫球。因此有  $16 / 2 = 8$  个高尔夫球。一半的高尔夫球是蓝色的。因此有  $8 / 2 = 4$  个蓝色高尔夫球。答案是 4。

**例 10.3** 提示设计中上下文信息的例子（灰色文字表示模型生成结果）

- **提示策略.** 针对不同的大语言模型设计合适的提示策略对于激发模型解决特定任务的能力非常重要。在某些情况下，添加特定的前缀或后缀有助于引导大语言模型解决复杂任务。例如，使用前缀“让我们一步一步地思考”可以激发大语言模型的逐步推理能力，而使用前缀“你是这项任务（或这个领域）的专家”可以提高大语言模型在某些特定任务（或领域）中的表现。此外，对于对话式的大语言模型（例如 ChatGPT），由于其使用了大量对话数据进行训练，因此更合适的做法是将提示拆分为多个子任务提示，以多轮对话的方法逐步输入给大语言模型。

**解答数学问题：**

**用户输入：**袋子里有 16 个球，其中一半是高尔夫球，并且所有高尔夫球中有一半是蓝色的球。请问有多少个蓝色高尔夫球？

**模型输出：**让我们一步一步地思考。

袋子里一共有 16 个球，其中一半是高尔夫球，也就是说，一共有 8 个高尔夫球。所有高尔夫球中有一半是蓝色的，也就是说有 4 个蓝色的高尔夫球。

**推荐旅游景点：**

**用户输入：**你是一名旅行向导。我会把我的位置告诉你，你需要向我推荐附近的旅游景点。我的第一个请求是“我在伊斯坦布尔，我只想参观博物馆”。

**模型输出：**当然，以下是一些您可以考虑参观的博物馆：

1. 伊斯坦布尔考古博物馆：这个博物馆位于苏丹艾哈迈德广场，是土耳其最重要的考古博物馆之一。它展示了来自不同历史时期的珍贵文物，包括古代近东、古希腊和古罗马的艺术品。
2. 伊斯坦布尔现代艺术博物馆：位于博斯普鲁斯海峡畔的卡拉库埃斯梅斯地区，这个博物馆致力于展示当代土耳其和国际艺术。它拥有丰富的永久收藏和临时展览，涵盖绘画、雕塑、摄影、视频艺术和装置艺术等多个领域。
3. 圣索菲亚博物馆：这座博物馆位于伊斯坦布尔历史中心的圣索菲亚大教堂内，展示了拜占庭和奥斯曼帝国时期的艺术和文物。您可以在这里欣赏到壮丽的拜占庭建筑和壁画，以及众多的历史文物。

**例 10.4** 提示设计中提示策略的例子**设计原则**

基于上述四个提示的关键要素，我们进一步总结了四项提示设计的基本原则，整体上有助于设计更为有效的任务提示。

- 清晰地表达任务目标. 模棱两可的任务描述很可能导致模型产生不准确甚至错误的回答。因此，在使用大语言模型时需要给出清晰明确的指令。具体来说，一个清晰详尽的任务描述中应当包含任务的各种要素信息，如任务目标、输入/输出数据（例如，“给定一份长文档，我希望你生成一份简明摘要”）和回复限制（例如，“摘要长度不能超过 50 个单词”）。通过提供清晰的任务描述，大语言模型可以更为有效地理解给定任务并生成所需的输出结果。

### 建议 (“清晰地表达任务目标” 原则的一些实用建议和样例)

- Make your prompt **as detailed as possible**, e.g., “Summarize the article into a short paragraph within 50 words. The major storyline and conclusion should be included, and the unimportant details can be omitted.”
- It is helpful to let the LLM know that it is **an expert with a prefixed prompt**, e.g., “You are a sophisticated expert in the domain of compute science.”
- Tell the model **more what it should do**, but not what it should not do.
- To avoid the LLM to generate too long output, you can just use the prompt: “Question: Short Answer: ”. Besides, you can also use the following suffixes, “in a or a few words”, “in one of two sentences”.
- If you want LLMs to provide the score for a text, it is necessary to provide a **detailed description about the scoring standard** with examples as reference.
- For few-shot chain-of-thought prompting, you can also use the prompt “Let’s think step-by-step”, and the few-shot examples should be **separated by “\n”** instead of full stop.
- The prompt should be **self-contained**, and better not include pronouns (e.g., it and they) in the context.
- When using LLMs for **comparing** two or more examples, the order affects the performance a lot.
- Before the prompt, **assigning a role for the LLM** is useful to help it better fulfill the following task instruction, e.g., “I want you to act as a lawyer”.
- For multi-choice questions, it is useful to **constrain the output space** of the LLM. You can use a more detailed explanation or just imposing constraints on the logits.
- For sorting based tasks (e.g., recommendation), instead of directly outputting the complete text of each item after sorting, one can **assign indicators** (e.g., ABCD) to the unsorted items and instruct the LLMs to directly output the sorted indicators.

● 分解为简单且详细的子任务. 该原则的目标是将一个复杂任务分解为若干个相对独立但又相互关联的子任务, 每个子任务都对应原始任务的某个方面或步骤。特别地, 我们可以显式地将子任务按编号列出 (例如, “通过依次执行以下任务形成一段连贯的叙述: 1. ...; 2. ...; 3. ...”)。这种策略有助于减少复杂任务的解决难度: 通过将复杂任务分解为若干个子任务并按照一定的顺序处理这些子任务, 模型能够逐步获得最终的答案。

## 建议（“分解为简单且详细的子任务”原则的一些实用建议和样例）

- For complex tasks, you can clearly describe the required intermediate steps to accomplish it, e.g., “Please answer the question step by step as: Step 1 - Decompose the question into several sub-questions, ...”
- When LLMs generate text according to some context (e.g., making recommendations according to purchase history), instructing them with the explanation about the generated result conditioned on context is helpful to improve the quality of the generated text.
- An approach similar to tree-of-thoughts but can be done in one prompt: e.g., *Imagine three different experts are answering this question. All experts will write down one step of their thinking, then share it with the group of experts. Then all experts will go on to the next step, etc. If any expert realizes they’re wrong at any point then they leave. The question is*
- As a symbol sequence can typically be divided into multiple segments (e.g.,  $i_1, i_2, i_3 \rightarrow i_1, i_2$  and  $i_2, i_3$ ), the preceding ones can be used as in-context exemplars to guide LLMs to predict the subsequent ones, meanwhile providing historical information.
- Let the LLM check its outputs before draw the conclusion, e.g., “Check whether the above solution is correct or not.”

• 提供少样本示例. 正如第 10.2 节所介绍的上下文学习方法, 在提示中加入少量目标任务的输入输出作为任务示例 (即少样本示例), 可以提升大语言模型解决复杂任务的能力。少样本示例有助于大语言模型在无需调整参数的前提下学习输入与输出之间的语义映射关系。在实践中, 我们可以根据目标任务专门为大语言模型设计若干高质量的示例, 这能够显著提升任务表现。

## 建议（“提供少样本示例”原则的一些实用建议和样例）

- Well-formatted in-context exemplars are very useful, especially for producing the outputs with complex formats.
- For few-shot chain-of-thought prompting, you can also use the prompt “Let’s think step-by-step”, and the few-shot examples should be separated by “\n” instead of full stop.
- You can also retrieve similar examples in context to supply the useful task-specific knowledge for LLMs. To retrieve more relevant examples, it is useful to first obtain the answer of the question, and then concatenate it with the question for retrieval.
- The diversity of the in-context exemplars within the prompt is also useful. If it is not easy to obtain diverse questions, you can also seek to keep the diversity of the

solutions for the questions.

- When using chat-based LLMs, you can decompose in-context exemplars into multi-turn messages, to better match the human-chatbot conversation format. Similarly, you can also decompose the reasoning process of an exemplars into multi-turn conversation.
- Complex and informative in-context exemplars can help LLMs answer complex questions.
- As a symbol sequence can typically be divided into multiple segments (*e.g.*,  $i_1, i_2, i_3 \rightarrow i_1, i_2$  and  $i_2, i_3$ ), the preceding ones can be used as in-context exemplars to guide LLMs to predict the subsequent ones, meanwhile providing historical information.
- Order matters for in-context exemplars and prompts components. For very long input data, the position of the question (first or last) may also affect the performance.
- If you can not obtain the in-context exemplars from existing datasets, an alternative way is to use the zero-shot generated ones from the LLM itself.

• 采用模型友好的提示格式。大语言模型采用专门构建的数据集进行预训练，因此可以从数据集中学习到大量的语言表达模式，发现并利用这些语言表达模式可以帮助我们更有效地使用大语言模型完成特定任务。对于提示中需要重点强调的部分，OpenAI 官方文档中建议用户可以使用特殊符号（例如 ###、三引号 “”””和””””、XML 标签等）进行分隔，从而让大语言模型更好地理解相关内容。此外，大多数现有的大语言模型主要在英语文本上进行训练，理解英语指令的能力更强，因此在执行任务时使用英语指令可能会获得更好的执行效果。对于非英语用户来说，通过机器翻译工具将非英语任务指令转换为英语指令再输入给大语言模型，可能会是一个更有效的策略。

#### 建议（“采用模型友好的提示格式”原则的一些实用建议和样例）

- For the question required factual knowledge, it is useful to first retrieve relevant documents via the search engine, and then concatenate them into the prompt as reference.
- To highlight some important parts in your prompt, please use special marks, *e.g.*, *quotation (""") and line break (\n)*. You can also use both of them for emphasizing, *e.g.*, `#### Complete sqlite SQL query only and with no explanation.\n #\n#### Sqlite SQL tables, with their properties: \n#\n{table}\n# {foreign_key}\n#\n#### {question}\n SELECT`.
- You can also retrieve similar examples in context to supply the useful task-specific

knowledge for LLMs. To retrieve more relevant examples, it is useful to **first obtain the answer** of the question, and then concatenate it with the question for retrieval.

- If the LLM can not well solve the task, you can **seek help from external tools** by prompting the LLM to manipulate them. In this way, the tools should be encapsulated into callable APIs with detailed description about their functions, to better guide the LLM to utilize the tools.
- OpenAI models can perform a task better in English than other languages. Thus, it is useful to first **translate the input into English** and then feed it to LLMs.
- For mathematical reasoning tasks, it is more effective to design specific prompts based on the format of programming language, e.g., “Let’s use python to solve math problems. Here are three examples how to do it,\n Q: Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?\n “def solution():\n """Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?""" \n money\_initial = 23\n bagels = 5\n bagel\_cost = 3\n money\_spent = bagels \* bagel\_cost\n money\_left = money\_initial - money\_spent\n result = money\_left\n return result”\n ..... \n How about this question?\n Q:”.

### 10.1.2 自动提示优化

人工设计提示虽然比较直接，但是需要耗费较多的人工成本，同时要求设计人员具有丰富的提示工程经验。此外，大语言模型对于提示设计比较敏感，人工设计的提示有时很难获得最优的效果，还可能导致任务性能的下降。因此，本小节将针对离散和连续这两种主要的提示形式，详细介绍提示的自动优化方法。需要注意的是，由于大语言模型参数量巨大，并且很多工作机制已经与传统预训练模型有着较大的差异，许多提示优化方法已经不再适用于大语言模型。然而，为了内容的完整性，本书仍然将这部分内容进行了收录。

#### 离散提示优化

离散提示通常是由一系列自然语言词元组成的完整句子表达（如“请根据提供的检索信息回答下列问题”）。然而，在离散的词元空间中进行组合搜索，不仅时间复杂度高，而且可能导致非最优的搜索结果。下面将介绍四种常见的离散提示优化方法，能够提升离散任务提示的有效性与搜索效率。

- 基于梯度的方法. 这类方法通过梯度更新技术以最大化模型的似然分数来优化离散提示的搜索过程。一种早期的代表性方法 [265] 使用梯度引导技术，首先将

提示初始化为一系列 “[MASK]” 标记，然后迭代地将提示中的词元替换为词典中的其他词元，通过词元替换产生的对数似然变化来近似估计梯度，进而为提示的每个位置贪心搜索出最佳的词元。由于该方法对提示的每个位置都进行所有候选词元的替换和梯度评估，因此需要模型进行多次前向和后向计算，导致搜索过程的效率较低。为了改进搜索效率，可以将离散词元转化为连续嵌入表示（又称为“软词元”），使用梯度直接对连续嵌入参数进行优化，最后将每个连续嵌入映射为词典中最邻近的离散词元。

- 基于强化学习的方法. 为了实现更有效的离散词元选择，另一种解决方法是将离散提示优化问题转换为强化学习问题，并使用强化学习算法进行求解。具体来说，可以将预训练语言模型作为强化学习中的策略网络并依次生成提示中的词元。在提示生成结束之后，策略网络可以获得任务特定的奖励信号，该奖励信号可通过强化学习算法用于策略网络参数的训练。在实践中，可以设计不同类型的奖励信号，比如真实标签与基于提示的预测标签是否一致、生成文本与给定条件的匹配程度。在最后的测试阶段，基于训练好的策略网络，可以采用贪心搜索策略来生成任务提示中的每个词元。

- 基于编辑的方法. 这类方法主要关注如何通过编辑现有的提示来改进模型的性能，通常是基于模型在目标任务上的表现来判断提示的好坏。它特别适用于无法直接访问模型内部状态（如梯度）的情况，例如只能通过 API 调用的模型。在这类方法中，通常需要事先定义好编辑操作，然后迭代地对提示进行修改，直至达到最大迭代轮次或者模型最佳性能。根据第 10.1.1 节的介绍，提示的关键要素包括任务描述、输入数据、上下文信息和提示策略。因此，常用的提示编辑操作有修改任务描述、添加或删除上下文任务示例、调整输入到输出的标签映射器（例如可以使用 “positive/negative” 或者 “正/负” 表示二分类）等。此外，提示编辑操作也可以根据不同的场景或者需求进行设计，以适配下游具体任务。整体流程可以概述如下：基于预定义的编辑操作，在现有提示的基础上修改得到新提示，并输入至模型得到目标任务上的表现，根据表现筛选出合适的提示。由于上述过程可能需要迭代进行，可以只选择少量测试样例来评估模型表现，以减少计算开销。

- 基于大语言模型的方法. 由于大语言模型具有通用的任务能力，因此可以将提示优化看作一个待求解的任务，进而直接使用大语言模型作为提示生成器来生成或改进提示 [266, 267]。基于大语言模型的自动提示生成框架将提示优化过程看作是一个由大语言模型指导的黑盒优化问题。该框架首先利用提示生成模型（用

于生成提示指令的大语言模型) 基于少量上下文示例生成一批候选的任务指令。随后, 使用“目标模型”(用于下游测试的大语言模型) 对这些候选指令在目标任务上的表现进行逐一评估。在评估过程中, 可以采用模型困惑度或任务准确率作为衡量指令质量的指标。上述过程可以通过基于蒙特卡洛搜索的多轮优化策略进行扩展。在每一轮迭代中, 根据模型表现对候选指令进行筛选得到高评分指令, 并利用大语言模型生成与高评分指令相似的新指令, 从而扩展候选指令集。迭代完成后, 选择模型表现最佳的候选指令作为最终使用的提示。然而, 上述方法没有充分考虑提示的整个历史改进轨迹, 因此可能在提示搜索过程中陷入局部最优或者产生效果震荡, 无法生成更好的提示。为了解决这一问题, 可以将所有改进的历史提示及其分数纳入提示优化过程, 以指导大语言模型逐步生成更好的新提示。

## 连续提示优化

与离散提示不同, 连续提示由一组连续空间中的嵌入向量组成, 可以根据下游任务的损失直接通过梯度更新进行优化。值得注意的是, 已有连续提示优化的工作主要是基于预训练语言模型开展的, 由于大语言模型参数量巨大, 连续提示受到的关注较为有限。已有的连续提示优化研究通常依赖于有监督学习方法。当数据稀缺的情况下, 还可以采用迁移学习方法来缓解目标任务标注数据不足的问题。下面将详细介绍这两种方法。

- **监督学习方法.** 这类方法将连续提示向量视为可训练的模型参数, 基于下游任务数据, 通过最小化交叉熵损失来优化连续提示。根据第 7.3 节中所讨论的内容, Prefix-tuning [212] 会在语言模型的每个 Transformer 层预置一串前缀 (即一组可训练的连续向量), 而 Prompt-tuning [213] 只会在输入层加入可训练的提示向量。通过固定语言模型的大规模参数而只微调这些连续的提示向量, 可以有效节省训练所需要的参数量。然而, 这些提示优化方法通常与输入无关, 缺乏对于输入语义的充分考虑。

- **迁移学习方法.** 有监督学习方法通常需要充足的训练数据来学习最优的任务提示, 很难在数据稀缺场景下获得较好的模型性能。为了解决这个问题, 基于提示的迁移学习方法首先为若干个具有代表性的源任务学习一个所有任务共享的连续提示, 然后使用该提示初始化目标任务的提示, 这可以为下游任务的提示优化提供良好的初始点。然而, 这种方法存在一定的局限性, 它在解决目标任务的所有实例时都使用了相同提示, 而即使是一个精心优化过的提示也未必适合所有的任务实例。为了解决这一问题, 可以为每个源任务独自学习任务特定的连续提示

(而不是所有源任务共享)，在解决目标任务的实例时，可以采用注意力机制等方式学习目标实例与每个源任务提示的相关性权重系数，对若干个源任务的提示向量进行加权组合，将组合后的新提示（为连续向量形式）用于帮助模型解决当前任务实例。

## 10.2 上下文学习

在 GPT-3 的论文 [23] 中，OpenAI 研究团队首次提出上下文学习（In-context learning, ICL）这种特殊的提示形式。目前，上下文学习已经成为使用大语言模型解决下游任务的一种主流途径。下面将详细介绍这一提示方法。

### 10.2.1 上下文学习的形式化定义

根据 GPT-3 论文中所给出的描述 [23]，上下文学习使用由任务描述和（或）示例所组成的自然语言文本作为提示。图 10.1 展示了上下文学习的提示构建过程。首先，通过自然语言描述任务，并从任务数据集中选择一些样本作为示例。其次，根据特定的模板，将这些示例按照特定顺序组合成提示内容。最后，将测试样本添加到提示后面，整体输入到大语言模型以生成输出。基于任务描述以及示例信息，大语言模型无需显式的梯度更新即可识别和执行新的任务。

形式上，我们使用  $D_k = \{f(x_1, y_1), \dots, f(x_k, y_k)\}$  来表示由  $k$  个样本构成的一组示例数据，其中  $f(x_k, y_k)$  是一个函数，负责将第  $k$  个任务样本转换为自然语言提示。给定任务描述  $I$ 、示例  $D_k$  以及新的输入  $x_{k+1}$ ，大语言模型生成答案  $\hat{y}_{k+1}$  的过程可以通过下面的公式来表述：

$$\underbrace{\text{LLM}}_{\text{大语言模型}} \left( \underbrace{I}_{\text{任务描述}}, \underbrace{f(x_1, y_1), \dots, f(x_k, y_k)}_{\text{示例}}, \underbrace{f(x_{k+1}, \quad)}_{\text{输入}}, \underbrace{\quad}_{\text{答案}} \right) \rightarrow \hat{y}_{k+1}. \quad (10.1)$$

值得一提的是，上下文学习与指令微调（详见第 7 章）之间存在着紧密的联系，因为它们都涉及将任务或样本转化为自然语言形式供大语言模型进行处理。在原始的 GPT-3 论文中，作者将上下文学习的提示定义为任务描述和示例的组合，这两部分均为可选。按照这个定义，如果大语言模型仅通过任务描述（即任务指令）来解决未见过的任务，也可以被看作是上下文学习的一种特例。两者的主要区别是，指令微调需要对大语言模型进行微调，而上下文学习仅通过提示的方式来调用大语言模型解决任务。此外，指令微调还可以有效提升大语言模型在执行目标

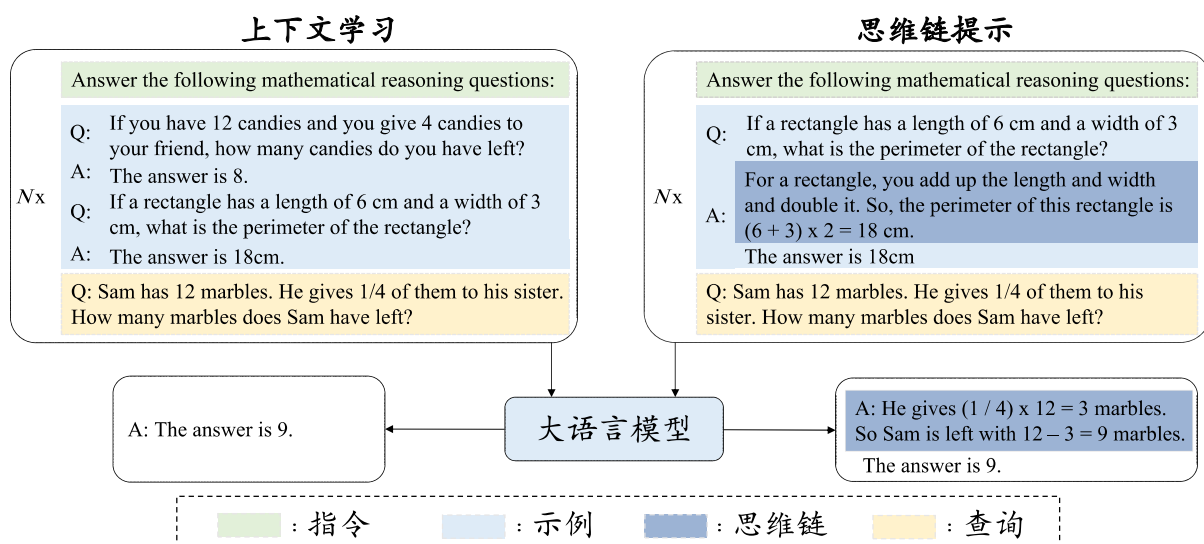


图 10.1 一个关于上下文学习和思维链提示的比较说明（图片来源：[10]）

任务时的上下文学习能力，尤其是在零样本场景下（即仅依赖任务描述而无需额外的示例）[41]。

### 10.2.2 示例设计

作为一个重要特点，上下文学习在提示中引入了示例数据，因此示例的选择和设计对于模型上下文学习的能力具有重要的影响。根据公式 10.1，我们将重点关注示例设计的三个关键因素，包括示例样本选择、样本格式化函数  $f(\cdot)$ ，以及示例排序策略。下面针对这三个方面展开详细讨论。

#### 示例选择

在上下文学习中，示例选择是一个关键步骤，其目的是为了从含有大量样本的集合中选取最有价值的示例，进而能够有效激发大语言模型在任务上的模型效果。下面介绍几种常见的示例选择方法，包括基于相关度排序的方法、基于集合多样性的方法和基于大语言模型的方法。

- **基于相关度排序的方法**. 在实际应用中，基于相关度排序的方法得到了广泛的应用，典型实现就是基于  $k$  近邻 ( $k$ -Nearest Neighbors,  $k$ -NN) 的相似度检索算法。该方法实现简单，而且能够有效地选择出与目标任务实例相关的示例。具体来说，可以使用文本嵌入模型（如 BERT）将所有候选样本映射到低维嵌入空间中，然后根据这些候选样本与测试样本的嵌入语义相似度进行排序，并选择出最相关的  $k$  个示例，最后将筛选出的示例作为上下文学习的示例集合。在实践中，这种

方法通常明显优于随机选择示例方法。

- 基于集合多样性的方法. 尽管  $k$  近邻检索算法简单易行，但是它通常独立地评估每个示例的相关性，而忽略了示例集合的整体效果。为了弥补这一不足，我们可以采取基于集合多样性的示例选择策略。这种策略旨在针对特定任务选择出具有代表性的、信息覆盖性好的示例集合，从而确保所选示例能够反应尽可能多的任务信息，从而为大语言模型的推理提供更丰富、更全面的信息。在选择过程中，除了考虑样本与目标任务的相关性，同时也要考虑与已选样本的相似性，需要综合考虑相关性与新颖性的平衡。在实现中，可以采用经典启发式的 MMR 算法 (Maximum Margin Ranking) 以及基于行列式点过程的 DPP 算法 (Determinantal Point Process)，从而加强示例集合的多样性。

- 基于大语言模型的方法. 除了上述两种方法外，另一个方法是将大语言模型作为评分器对候选样本进行评估，进而选择出优质的示例。一种最直接的评估方法是通过计算在加入当前示例后大语言模型性能的增益来评估示例的有效性，以此筛选出有效的示例。但是，这种方法需要大语言模型进行重复多次计算，才能选择出最优的示例集合。为了减少大语言模型评估的开销，还可以根据大语言模型的评分结果选择出少量的正负示例用于训练一个分类器，该分类器通过正负示例能够学习到如何准确地区分和筛选出高质量示例，从而更准确地来指导后续的示例选择过程。

总体来说，在上下文学习中进行示例选择时应确保所选示例包含丰富的任务信息且与测试样本保持高度的相关性 [268]。

## 示例格式

示例模版（即公式 10.1 中的示例格式化函数  $f(\cdot)$ ）对大语言模型的性能有着非常重要的影响。通常来说，可以使用两种主流的方法进行示例格式的构建，包括人工标注的示例格式与自动生成的示例格式。

- 人工标注的格式. 人工标注是构建高质量任务示例格式的一种常用方式。在实现中，首先需要定义好输入与输出的格式，然后添加详细的任务描述，以帮助大语言模型更好地理解当前示例所要表达的任务需求。例 10.5 展示了人工标注的示例格式的例子。最简单的示例格式只需要显式标识出输入与输出，让大语言模型自动学习到输入与输出之间的语义映射关系。进一步，在提示内部加入相关的任务描述有助于模型更精准地理解任务的要求，从而生成更准确的答案。最后，为了更好地激发大语言模型的推理能力，可以在输出中加入思维链，展示模型的逐

步推理过程。人工标注的示例格式的优势在于格式清晰且易于理解，但在处理大量任务时，可能面临多样性不足的问题。

**(1) 包含输入与输出的示例格式：**

**输入：**罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？

**输出：**11。

**示例模板：**问题：{输入} 答案：{输出}

**具体示例：**问题：罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？答案：11。

**(2) 增加任务描述的示例格式：**

**输入：**罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？

**输出：**11。

**示例模板：**下面是一个小学数学问题。问题：{输入} 答案：{输出}

**具体示例：**下面是一个小学数学问题。问题：罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？答案：11。

**(3) 增加思维链的示例格式：**

**输入：**罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？

**输出：**让我们一步一步地思考。罗杰一开始有 5 个球，2 罐每罐 3 个网球就是 6 个网球。 $5 + 6 = 11$ 。因此答案是 11。

**示例模板：**下面是一个小学数学问题。问题：{输入} 答案：{输出}

**具体示例：**下面是一个小学数学问题。问题：罗杰有 5 个网球，他又买了 2 罐网球，每罐有 3 个网球。他现在有多少个网球？答案：让我们一步一步地思考。罗杰开始有 5 个球，2 罐每罐 3 个网球就是 6 个网球。 $5 + 6 = 11$ 。因此答案是 11。

### 例 10.5 人工标注的示例格式

• 自动生成的格式. 为了缓解人工标注的局限性，还可以采用大语言模型自动生成示例格式。这种方法的核心在于借助大语言模型的上下文学习能力，进而大规模扩充新任务的示例模版。具体来说，首先人工标注一部分的示例模板作为种子集合加入到大语言模型的输入中。然后，利用大语言模型强大的少样本学习能力，指导其为新任务生成相应的示例模版。最后，对这些生成的示例模版进行筛选与后处理，使之符合任务要求。例 10.6 展示了大语言模型根据少量人工标注的

模板来自动生成示例格式的例子。通过提供示例的输入、输出与指令，大语言模型能够根据新任务的输入和输出为其生成对应的指令。

请根据输入输出自动撰写一段指令：

**示例输入：** Sentence: This house is surprisingly not constructed very well, and you probably need more money to fix it after you buy it. If you ask me, I would suggest you to consider other candidates.

**示例输出：** This house does not seem to be constructed well, so you may need to spend more money to fix it after you purchase it. I would suggest that you look at other properties.

**示例指令：** Suggest a better and more professional rephrasing of the following sentence.

**示例输入：**

Application Form:

Name:\_\_\_ Age:\_\_\_ Sex:\_\_\_

**示例输出：** Name: John Doe. Age: 25. Sex: Male

**示例指令：** I am looking for a job and I need to fill out an application form. Can you please help me complete it?

**示例输入：** [10, 92, 2, 5, -4, 92, 5, 101]

**示例输出：** [-4, 2, 5, 5, 10, 92, 92, 101]

**示例指令：** Sort the given list ascendingly.

**输入：** Address: 123 Main Street, City: San Francisco

**输出：** 94105

**指令：** Given an address and city, come up with the zip code.

### 例 10.6 自动生成的示例格式（灰色文字表示模型生成指令）

## 示例顺序

在上下文学习中，大语言模型往往会受到位置偏置的影响，表现为对示例顺序具有一定的敏感性。因此，设计合理的示例顺序也是上下文学习中需要考虑的一个问题，旨在为所选择的示例找到最有效的排序方式以提升模型性能。确定大语言模型的最优的示例顺序通常分为两个步骤：产生示例的候选顺序和评估示例顺序的有效性。

- 产生候选示例顺序。在第一个步骤中，我们需要为所选择的示例产生候选

排序。一种最直接的方法是枚举给定示例的所有可能排列组合，然后从中随机选取一种排列作为示例的顺序。然而，这种随机选择方法所产生的结果具有较大的方差，可能导致模型性能的不稳定。鉴于大语言模型在做出预测时，倾向于依赖于提示末端的信息。另一种更常用的方式是根据示例与测试样本之间的语义相似度进行排序，然后将与测试样例相似度更高的示例放在更靠近测试样本的位置。这种方法可以加强大语言模型在推理过程中对于语义相关的示例进行利用，从而提升模型性能。

- 评估示例顺序质量. 在示例顺序确定之后，下一步是评估这一顺序的质量。在测试集样本可获得的情况下，可以直接测试大语言模型基于该示例顺序的任务性能，以此作为当前示例顺序的评分。然而，在许多情况下，我们可能无法获得测试样本，因此需要人工创建独立的验证集进行示例顺序的评估。另一种不依赖测试数据的评估方法是采用模型对于预测结果的不确定性作为评估指标。具体来说，可以计算基于该示例顺序大语言模型预测分布的熵值，选择熵值较低的示例顺序作为较为有效的顺序。熵值越低，意味着模型预测分布越不均匀，则模型预测的置信度更高。

### 10.2.3 底层机制

上下文学习能力是一种具有代表性的大语言模型能力。这种通过示例进行学习的范式虽然在传统机器学习模型中也有涉及（例如  $k$  近邻分类器），但是整体的应用范围与任务场景非常局限。特别是，大语言模型完全通过提示设计来进行上下文示例的学习，这其中的内在原理与工作机制值得深入思考。在本节中，我们将深入探讨与大语言模型上下文学习能力紧密相关的两个核心问题：一是预训练阶段如何影响上下文学习能力，二是生成阶段大语言模型如何支持上下文学习。

#### 预训练阶段对上下文学习能力的影响

预训练阶段主要有两个关键因素对大语言模型上下文学习能力产生影响：预训练任务和预训练数据。这两方面分别关注如何设计训练任务和如何选择训练数据来提升模型的上下文学习能力。

- 预训练任务. 上下文学习的概念最初在 GPT-3 的论文 [23] 中被正式提出，论文通过相关实验发现上下文学习能力随着模型规模的增大而增强。随着预训练技术的改进，后续研究发现，即使是小规模模型，通过设计专门的训练任务（如根据示例和输入预测标签），进行继续预训练 [269] 或微调 [270]，也能够获得上下

文学习能力，甚至在某些情况下可能超越规模更大的模型。这表明预训练任务的设计对于上下文学习能力的习得具有重要的影响。具体来说，MetaICL [270] 认为，通过元训练任务可以让模型自动学习到如何通过输入中的少量示例重构任务信息，进而更有效地完成上下文学习。因此，MetaICL 使用了数十个不同领域的 NLP 任务作为元训练任务。对于每一个元训练任务，抽取出若干样本作为示例（其余样本则用于训练），大语言模型采用上下文学习的方式进行训练，即根据示例和待预测样本的输入预测对应的输出。这种训练方式使得无需在输入中提供任务描述，只需提供少量示例，大语言模型即可学会解决对应的目标任务。实验结果显示，元训练任务越多且越多样，模型的上下文学习能力就越强，这也证明了预训练任务对于大语言模型上下文学习能力具有很大的影响。

- 预训练数据. 除了训练任务外，预训练数据的选择对上下文学习能力也有显著影响，`dan` 并非所有的预训练数据都对上下文学习能力的提升同等重要。研究发现，通过混合不同领域的预训练数据，增强预训练语料的多样性可以提高大语言模型的上下文学习能力 [271]。此外，预训练数据的长程依赖关系也是改善模型上下文学习能力的重要因素。通过将前后相关的文本直接拼接进行训练，模型能够更好地理解文本之间的关联性，从而提升上下文学习的能力 [272]。进一步，为了筛选出对模型上下文学习能力有重要影响的训练数据，研究人员通过计算预训练数据和上下文学习的测试数据的梯度之间的相似度，可以得到具有较高相似度的训练数据子集 [273]。实验发现，这些具有高相似梯度的预训练数据中包含了更高密度的低频长尾词汇，模型对这部分数据的学习难度较高，因此可能有助于提升模型上下文学习的能力。

### 推理阶段大语言模型执行上下文学习的方式

在推理阶段，由于上下文学习不涉及显式的学习过程或参数更新，因此可以主要关注在给定示例的情况下，大语言模型如何执行上下文学习。大语言模型使用示例数据的方式主要分为两种范式，包括任务识别和任务学习 [274]。在任务识别范式中，大语言模型通过分析示例来理解并识别需要执行的任务；而在任务学习范式中，大语言模型则尝试从示例中提取正确的信息来完成任务。下面对于这两种方式进行具体介绍。

- 任务识别. 大语言模型具备从所提供示例中辨识当前任务的能力，并能利用其在预训练阶段所积累的丰富先验知识来解决这些任务，这一范式不受示例的输入和输出映射的影响。基于概率近似正确（Probably Approximately Correct, PAC）

的理论框架认为预训练数据中存在能够表征各种任务信息的隐变量。因此在上下文学习中，大语言模型具备从给定示例中学习并编码这些隐变量的能力，因此能够通过上下文示例实现任务的自动识别和适应 [275]。随后，大语言模型根据这个任务隐向量的指导，在接收到新的输入时自动触发相应的任务识别过程，并生成符合任务要求的输出 [276, 277]。

- **任务学习.** 第二种观点认为大语言模型还具备通过示例数据学习预训练阶段未涉及的新任务的能力。这种观点主要从梯度下降的角度来分析上下文学习的机理，并将其视为一种隐式的微调过程 [278, 279]。具体来说，从隐式梯度下降的角度分析，上下文学习机制可以被分解为以下两个步骤。首先，大语言模型通过前向计算过程，针对给定示例生成相应的元梯度（类似于梯度下降时的梯度，但没有显式计算的过程）。然后，模型利用注意力机制隐式地执行了梯度下降。这一过程类似于传统机器学习中的参数更新，但不同之处在于它是在模型的前向传播过程中隐式完成的，无需显式的参数更新。除了从梯度下降的角度进行解释，上下文学习还可以被抽象为模型内部的一种更复杂的算法学习过程 [280]。具体来说，在预训练阶段，大语言模型通过其参数编码了一个隐式模型。因此，在上下文学习的前向计算阶段，借助上下文学习中的示例引导，大语言模型能够通过诸如决策树等更复杂的学习算法来更新其内部的隐式模型。

在上下文学习中，现有的文献通常认为大语言模型能够同时展现出任务识别和任务学习两种能力，但是这两种能力的强弱与模型的规模紧密相关。其中，规模较小（如 350M 参数）的模型已经能展现出较强的任务识别能力，能够简单地识别任务的类型和要求 [274]；而任务学习能力要求模型从示例中学习全新的任务解决方案，通常较大规模（如 66B 参数）的模型能展现出更强的任务学习能力 [274]。一项研究通过将上下文示例的真实标签替换为“随机标签”或者“随机符号”，分别对模型的任务识别和任务学习能力进行探究 [281]。例如，在情感分析任务中，从标签空间中均匀随机采样示例的标签（“positive/negative”），模型只需要进行任务识别；而使用没有明确语义含义的符号如“foo/bar”替换真实标签“positive/negative”，迫使模型不依赖其先验知识，而是必须从提供的示例中学习新的标签映射以解决当前任务。实验结果显示，当使用颠倒或语义不相关的符号作为标签时，规模较大的模型的性能下降更小。这意味着大模型能够更好地分析和学习给定的示例信息，从示例中学习出标签信息的对应关系，进而采用学习到的策略完成任务。

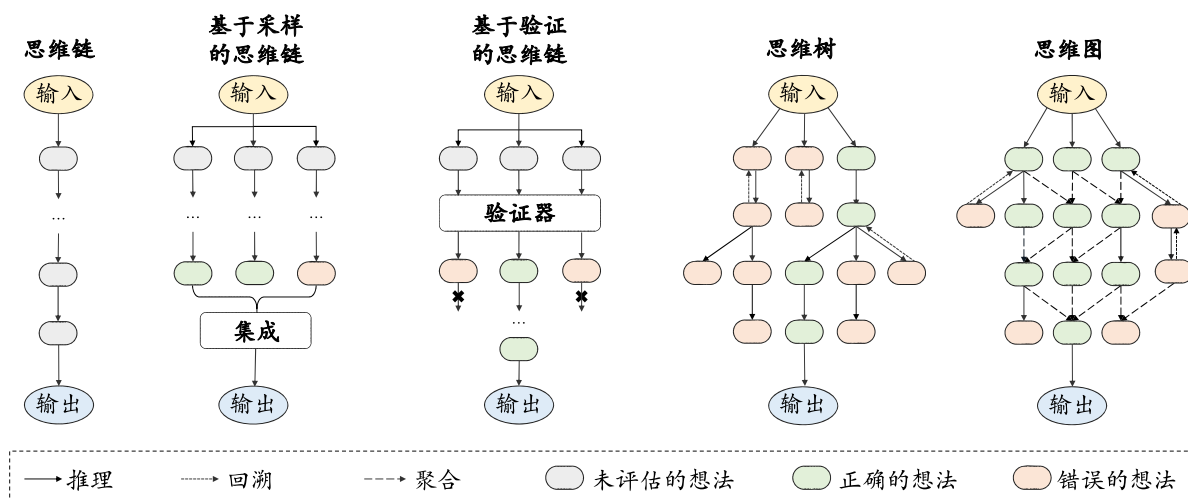


图 10.2 思维链提示技术的演化过程（图片来源：[10]）

## 10.3 思维链提示

思维链提示 [25, 282] 是一种高级提示策略，旨在增强大语言模型在各类复杂推理任务上的表现。常见的推理任务包括算术推理 [283]、常识推理 [284] 以及符号推理 [25] 等多种任务。与上下文学习方法仅使用〈输入，输出〉二元组来构造提示不同，思维链提示进一步融合了中间的推理步骤来指导从输入到输出的推理过程。图 10.1 展示了一个思维链提示的具体例子。在本节中，我们将介绍基础的思维链提示方法以及相关的增强策略，还将探讨思维链的能力来源以及思维链提示对模型推理的影响。

### 10.3.1 思维链提示的基本形式

思维链提示作为上下文学习的一种扩展形式，将原始的〈输入，输出〉映射关系转换为〈输入，思维链，输出〉这一三元组形式。在这个结构中，思维链扮演着重要的角色，它提供了一系列语义连贯且具有逻辑性的中间推理步骤，有效地建立起输入与输出之间的桥接关系。在思维链提示的作用下，大语言模型可以根据输入生成对应的思维链及答案。然而，与简单的〈输入，输出〉二元组相比，思维链的获取通常更为复杂，往往需要借助人工标注的方式。目前有一些简单的方法可以让大模型在回答问题之前生成思考过程。例如，通过向大语言模型提供诸如“Let’s think step by step.” [285] 或“Take a deep breath and work on this problem step-by-step.” [267] 这样的诱导性指令，能够在不提供思维链示例的情况下，仍然

让大语言模型先生成思维链再回答问题来提高准确率。图 10.1 右侧的例子展示了大语言模型在思维链提示的作用下，一步步生成中间推理步骤，最终得到了正确的答案。

### 10.3.2 思维链提示的优化策略

尽管大语言模型在基本的思维链提示策略下已经在推理任务中展现出一定的性能提升，但仍然存在推理过程错误、生成答案不稳定等问题。针对这些问题，本节将从下列三个方面探讨如何对基础的思维链提示方法进行改进：针对输入端对大模型的思维链示例进行增强（增强的思维链示例设计）、针对大模型的思维链生成过程进行改进（高级的思维链生成方法）以及针对整个思维链结构进行优化（拓展的推理结构）。图 10.2 展示了代表性的思维链提示策略的演变历程。

#### 思维链示例设计

目前大语言模型在使用思维链提示进行推理时，大多采用了上下文学习的设定，即思维链提示通过示例的形式输入给大语言模型。因此，接下来将介绍两种常用的在上下文学习场景下的思维链示例设计方法。

- 复杂化的思维链. 基于复杂度指标设计思维链示例是一种简单有效的策略。思维链复杂化的体现主要在于推理步骤的增多。由于每一个推理步骤都可以看作是一个子问题的解答，因此更多的推理步骤也包含了对于更多子问题的解答，推理过程也更加缜密。当使用较多推理步骤的示例作为提示输入给模型时，模型更容易学习到多种子问题的解决方案以及对应的逻辑推理过程，能够提升模型在复杂推理任务上的表现。除了将推理步骤的数目作为复杂度指标，还可以使用问题长度对问题的复杂度进行量化。问题越长说明其包含更多的输入信息，则问题求解可能需要更多的推理步骤。对于某些没有人工标注思维链的数据集，可以选择最长的若干问题，然后对这些问题的思维链进行人工标注作为思维链示例。基于这些复杂思维链示例，模型通常可以获得相较于随机选择思维链示例更好的性能。

- 多样化的思维链. 除了设计更为复杂的思维链示例，在提示中包含多样化的思维链示例能够有效改善模型的推理能力，主要是因为多样化的思维链示例可以为模型提供多种思考方式以及推理结构。为了选择出多样化的思维链，可以首先利用聚类算法（例如  $k$ -means 聚类）将训练集中的问题划分为  $k$  个簇（ $k$  为所需的示例数量），簇内部的问题比较相似，而不同簇的问题差别较大。然后，预定义一系列启发式规则，从每个簇中选择距离质心最近且满足规则的问题作为该簇的代表。

表性问题，将该问题输入给大语言模型并生成对应的思维链和答案作为示例。由于每个问题来自于不同的簇，从而保证了示例的多样性。实验发现，虽然大模型生成的思维链示例可能存在错误，但是当选择更加多样化的示例时，思维链示例中的错误对模型性能的影响会显著降低 [286]。

## 思维链生成方法

在上述内容中，我们介绍了如何在模型的输入侧对思维链示例进行增强。另一方面，模型在生成思维链时容易出现推理错误和生成结果不稳定等情况，还需要对大语言模型生成思维链的过程进行改进。本部分将重点介绍两种改进思维链生成过程的方法：基于采样的方法与基于验证的方法。

- 基于采样的方法. 大语言模型在使用单一的思维链进行推理时，一旦中间推理步骤出错，容易导致最终生成错误的答案。为了缓解这一问题，可以通过采样多条推理路径来缓解单一推理路径的不稳定问题。作为一种代表性方法，Self-consistency [287] 首先使用大语言模型生成多个推理路径和对应的答案（如图 10.2 所示），然后对于这些候选答案进行集成并获得最终输出。具体的集成方法可以选择各条推理路径所得到答案中出现频率最高的那个答案作为最终输出，在某些情况下也可以对所有答案进行某种形式的加权。我们还可以对上述过程做进一步的扩展：假设大语言模型在一个思维链提示下生成了  $M_1$  条推理路径，那么可以使用  $M_2$  个思维链提示依次输入给大语言模型，这样一共就能得到  $M_1 \times M_2$  条推理路径，从中投票选出最终的答案，进一步增加答案的可靠性。基于采样的思维链生成方法不仅简单易行，而且相较于单一思维链方法在多个任务中展现出了更为优异的性能。然而，在一些特定的任务场景中，当仅使用单一推理路径时，模型使用思维链提示的效果可能不如基础提示的效果。例如，对于句子的情感分类任务，由于问题过于简单，加入思维链提示之后反而会使模型过度思考，从而得出错误的答案。

- 基于验证的方法. 思维链提示所具有的顺序推理本质可能导致推理过程中出现错误传递或累积的现象。为了解决这一问题，可以使用专门训练的验证器或大语言模型自身来验证所生成的推理步骤的准确性。下面以 DIVERSE 方法 [288] 为例进行具体介绍。DIVERSE 分别训练了针对整个推理路径和中间推理步骤的验证器，从不同的粒度实现更为全面的检查。针对整个推理路径的验证器通过如下方法训练得到：首先选择一个包含大量问题答案对的数据集，然后将问题输入给大语言模型，通过思维链提示的方法使其生成推理路径和最终答案。如果模型生成

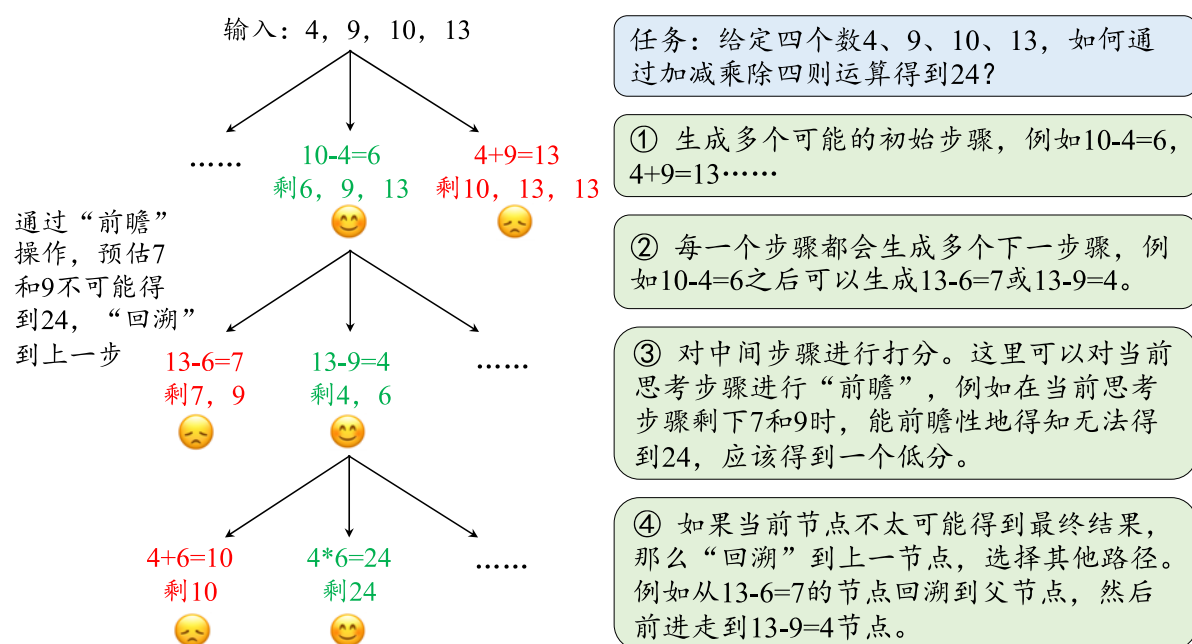


图 10.3 大语言模型使用思维树方法解决 24 点游戏

的答案和数据集标注的答案一致, 则判为正例, 否则判为负例。最后使用构造的〈问题, 推理链, 答案〉数据训练一个二分类器, 从而可以对任意一个推理路径进行打分。训练针对中间步骤的验证器也可以采用类似的方案。然而, 与整体推理路径的数据标注相比, 构造面向中间步骤的正负例数据更加困难。这里可以采取一个简化处理: 对于每一个训练集中的问题, 我们采样多次得到多个推理路径, 对于得出正确答案的推理路径, 中间的每一个步骤我们都认为是正确的, 作为正例; 对于得出错误答案的推理路径, 如果其中某个步骤和正例的推理路径相一致, 也认为是正例, 否则作为负例。通过这样构造出来的数据, 用同样的方法训练一个二分类器, 从而可以对模型输出的中间步骤进行打分。

### 拓展的推理结构

尽管基本的思维链提示具有广泛的适用性, 但是所采用的链式推理结构在处理较为复杂的任务时 (例如需要进行前瞻和回溯探索) 仍然存在一定的局限性。为了突破链式推理结构的限制, 可以将思维链的结构进一步拓展, 从而获得更强的推理性能。

- 树形结构的推理. 考虑到许多推理任务需要模型前瞻和探索可能的解决方案, 可以将推理过程刻画为一个层次化的树形结构, 进而问题的求解就转化为在树上的搜索问题。这一方法的代表性工作是思维树 (Tree of Thought, ToT) [289, 290]。思维树的每个节点对应一个思考步骤, 父节点与子节点之间的连边表示从一

个步骤进行下一个步骤。它和思维链的区别在于：思维链从一个节点出发，只能生成一个节点，而思维树则可以生成多个节点。当某一个思考步骤无法得到正确答案时，可以回溯到它的父节点，选择另一个子节点继续推理。图 10.3 以 24 点游戏为例介绍如何使用思维树解决问题<sup>2</sup>。其中 ① 和 ② 与思维链是一样的，最关键的步骤是 ③ 和 ④。对于思维链来说，只有走到最后一步才能判断当前的推理路径是否正确，如果出现错误只能从头开始推理，这极大降低了推理效率。但是 ③ 通过前瞻性判断，能够预估当前节点得到最终答案的可能性并给出一个评分。这样，在 ④ 的搜索算法中，我们可以提前放弃一些不太可能得到最终答案的路径（对应评分较低的节点），而优先选择那些评分更高的推理路径进行下一步的推理。图 10.2 将思维树与其他推理结构进行了对比。

- 图形结构的推理. 相较于树形结构，图形结构能够支持更为复杂的拓扑结构，从而刻画更加错综复杂的推理关系，可以展现出更强的推理性能。这一方法的代表性工作是思维图（Graph of Thought, GoT）[291, 292]。思维图将整个推理过程抽象为图结构，其中的节点表示大语言模型的中间步骤，节点之间的连边表示这些步骤之间的依赖关系。由于树形结构中只有父节点和子节点之间有连边，因此无法构建不同子节点之间的联系。思维图则允许图上的任意节点相连，因此可以在生成新的中间步骤的同时考虑其他推理路径。图 10.4 以含有重复数字的 0~9 数组排序为例介绍大语言模型如何使用思维图解决排序问题。在这个场景下，大语言模型难以对长数组进行准确地排序，但是短数组排序对于模型来说更为简单。因此，思维图方法首先将输入数组分成 4 组，分别进行排序，然后再对结果进行合并。思维图和思维树的区别在于，思维树的子节点只能进行前向搜索和回溯，而思维图的子节点可以和其他子节点进行汇聚，得到新的中间步骤，然后进行下一步的推理。图 10.2 将思维图与其他推理结构进行了对比。

### 10.3.3 关于思维链的进一步讨论

作为一种重要的“涌现能力”，思维链提示能够显著提升大语言模型的推理能力。但是在一些简单任务上，思维链提示有时甚至会带来效果上的下降。因此，一个值得探讨的问题是：为什么思维链提示能显著提升大语言模型在推理任务上的效果？下面我们从训练阶段思维链推理能力的来源、测试阶段思维链对模型的影响两个角度进行讨论。

<sup>2</sup>给定 4 个数，通过四则运算得到 24

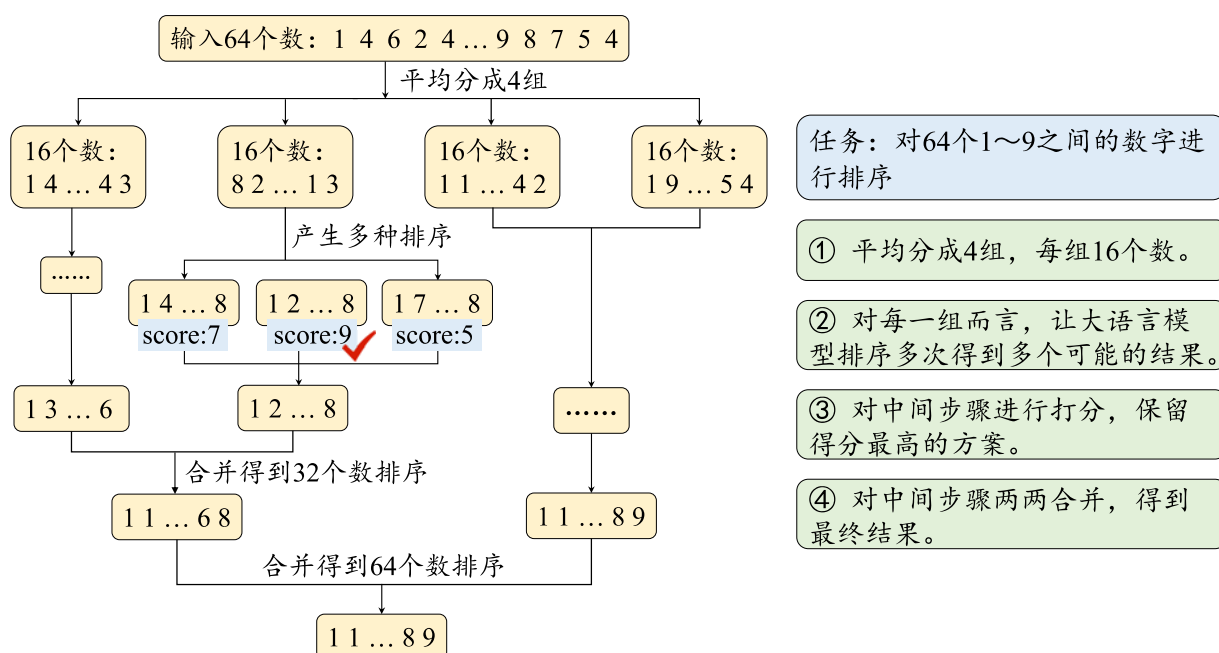


图 10.4 大模型使用思维图方法解决数组排序

● 思维链推理能力的来源. 对于思维链工作机制的研究需要探究推理的本质。斯坦福大学的研究人员 [293] 假设思维链对大语言模型有效的原因是训练数据中存在很多相互重叠且互相影响的局部变量空间（例如主题、概念和关键词等）。在这个条件下，即使两个变量没有在训练数据中共现，也可以通过一系列重叠的中间变量的推理而联系起来。为了验证这一假设，研究人员构建了一个具有链式结构的贝叶斯网络，并用这个网络合成了一批训练样本，这些样本包含许多相互影响的局部变量空间。然后，使用这批数据来训练一个语言模型，根据给定一个变量来预测另一个变量的条件概率。实验结果发现，如果这两个变量不经常在数据中共现时，模型直接预测这个条件概率总会与真实概率有一定偏差，但是当使用中间变量进行推理预测时，可以获得比直接预测更小的偏差；而当这两个变量经常在数据中共现时，通过中间变量推理和直接预测两种方式带来的偏差会比较接近。还有研究工作从函数学习的角度出发 [294]，认为复杂推理任务可以看作是一种组合函数，因此思维链推理实际上是将组合函数的学习过程分解为了两个不同阶段：信息聚焦和上下文学习单步的组合函数。在第一阶段，语言模型将隐式地聚焦到思维链提示中与推理的中间步骤相关的信息。在第二阶段，基于聚焦得到的提示，语言模型通过上下文学习输出一个推理的步骤（即单步组合函数的解），并走向下一步，从而得到最终答案（即整个组合函数的最终解）。通过理论证明与实验验证，研究人员发现信息聚焦阶段显著减少了上下文学习的复杂度，只需要

关注提示中与推理相关的重要信息，而上下文学习阶段则促进了复杂组合函数的学习过程，而标准提示则很难让模型学习到这种复杂函数。

- 思维链提示对模型推理的影响. 为了研究思维链提示对模型推理能力的影响，主要通过对思维链提示进行扰动，然后观察模型行为上的变化来得出相应的结论。谷歌的研究人员 [295] 将思维链分成两部分：符号（例如数学题中的数字、常识问答中的实体）和模式（例如数学题中的算式、常识问答中的句子结构和模板），分别研究它们对模型推理能力的影响。实验结果发现，不管是符号还是模式，其作用都主要体现为表达任务意图，而具体的内容并不重要，重要的是它们与问题的相关性以及推理过程的逻辑性 [296]。进一步，在某些数学推理任务上（少样本学习），思维链示例中算式的正确与否甚至不会显著影响模型的性能，这些符号与模式的作用更多是体现为对于任务目标的表达。研究人员认为思维链可以看作一种增强的上下文学习：因为任务过于复杂，基础提示已经无法准确表达任务意图，因此需要思维链提示来增强对于任务意图的表达。也有研究发现 [297]，即使不对语言模型使用思维链提示，只要其生成的文本中包含显式的推理过程，也能显著改善模型的推理能力。具体来说，当语言模型生成第一个词元时，采样出前  $k$  个概率最大的词元，然后继续解码，生成  $k$  条可能的文本序列。在这些生成的文本序列中，某些会包含推理路径，而这些具有推理路径的文本序列产生的正确答案概率显著高于其他路径。这表明，思维链提示通过激发模型生成中间推理步骤来提高其生成正确答案的概率。